Introduction to Microcontrollers

Facilitator Guide

Prepared by BLIS GLOBAL Robotics Department

Copyright @2024

Introduction

This document is showing the list of experiments and procedures which will be conducted by students in this course

As Facilitator you should be in advance in the following topics:

Week 3: Basic Arduino Programming

1. Introduction to Basic Programming Concepts

When programming an Arduino, understanding some key concepts such as **variables**, **data types**, and the basic **structure of the program** is essential. This section will help you become familiar with the building blocks of Arduino programming.

2. Variables and Data Types

A **variable** is a container that stores information that can be used and modified in a program. Variables are essential in any programming language as they allow us to store and manipulate data.

Declaring a Variable:

To declare a variable in Arduino, we first define its **data type** and then give it a **name**. For example:

```cpp
int ledPin = 13;   // Declare an integer variable 'ledPin' and set its value to 13
```

In the example above:

- int is the **data type**, which stands for integer (a whole number).
- ledPin is the name of the variable.
- 13 is the value assigned to ledPin.

Common Data Types in Arduino:

- int: Represents an integer, which is a whole number (e.g., 0, 42, -100).
- float: Represents a number with decimals (e.g., 3.14, -0.5).

- char: Represents a single character (e.g., 'A', 'B').
- boolean: Represents true or false values.

Example:

```cpp
int sensorValue = 0;  // Integer variable to store sensor reading
float temperature = 36.5;  // Float variable to store temperature reading
boolean isOn = false;  // Boolean variable to store whether something is on or off
```

3. Basic Syntax (setup() and loop() Functions)

The structure of an Arduino program consists of two main parts: the setup() function and the loop() function.

setup() Function:

The setup() function is where you place any code that you want to run **once** when the Arduino starts up. It is often used to configure pin modes and initialize variables or sensors.

```cpp
void setup() {
  pinMode(13, OUTPUT);  // Set pin 13 as an output pin
}
```

loop() Function:

The loop() function contains the code that runs **repeatedly** after the setup() function has been executed. Any actions that you want to perform continuously should be written inside the loop() function.

```cpp
void loop() {
  digitalWrite(13, HIGH);  // Turn on LED
  delay(1000);  // Wait for 1 second
  digitalWrite(13, LOW);   // Turn off LED
  delay(1000);  // Wait for 1 second
}
```

In the example above, the Arduino will turn on the LED connected to pin 13, wait for 1 second, turn off the LED, and then wait another second. This process repeats indefinitely.

4. Digital I/O (Input/Output)

Digital I/O is one of the core features of the Arduino. It allows you to control devices and read signals from the world around you, like turning on LEDs or reading from a button.

Digital Pins:

The digital pins on an Arduino can either be set as **INPUT** or **OUTPUT**. These pins are used to send or receive digital signals (either HIGH or LOW).

- **HIGH**: Represents a voltage of 5V (or 3.3V on some boards).
- **LOW**: Represents a voltage of 0V (ground).

Setting Digital Pins:

You use the pinMode() function to set a pin as either an input or an output. For example:

```cpp
pinMode(13, OUTPUT);  // Set pin 13 as an output pin (to control an LED)
pinMode(7, INPUT);    // Set pin 7 as an input pin (to read a button)
```

5. Writing to Digital Pins

To send a signal to a digital pin, you use the digitalWrite() function. This function can set a pin to either HIGH (5V) or LOW (0V).

Example of Turning On/Off an LED:

```cpp
digitalWrite(13, HIGH);  // Turn on the LED
delay(1000);             // Wait for 1 second
digitalWrite(13, LOW);   // Turn off the LED
delay(1000);             // Wait for 1 second
```

6. Reading from Digital Pins

To read the state of a digital pin (either HIGH or LOW), you use the digitalRead() function. This is useful for reading the state of buttons, switches, or sensors.

Example of Reading a Button:

cpp Copy code

```cpp
int buttonState = digitalRead(7);  // Read the state of the button connected to pin 7
if (buttonState == HIGH) {
  // If the button is pressed, turn on the LED
  digitalWrite(13, HIGH);
} else {
  // Otherwise, turn off the LED
  digitalWrite(13, LOW);
}
```

In this example, we read the state of a button connected to pin 7. If the button is pressed, the LED connected to pin 13 turns on.

Summary

- **Variables**: Used to store data (e.g., integers, floats, booleans).
- **setup() and loop()**: The two main functions in an Arduino program. setup() runs once, and loop() runs repeatedly.
- **Digital Pins**: Used for reading inputs (e.g., buttons) and controlling outputs (e.g., LEDs).
- **digitalWrite()**: Sends a HIGH or LOW signal to a pin.
- **digitalRead()**: Reads the state (HIGH or LOW) of a pin.

By understanding these fundamental concepts, you are well on your way to mastering Arduino programming and using it to create interactive projects.